

Audix micro-ROS Migration

This workspace now has a micro-ROS path for the ESP32 base controller. The old ESP UART JSON bridge is no longer the default launch path. The RX/TX serial link is used by `micro_ros_agent`, and the ESP32 runs a micro-ROS client in its own FreeRTOS task.

Architecture

```
Raspberry Pi ROS 2
  /micro_ros_agent
    serial RX/TX to ESP32

  /audix/micro_ros_base
    converts ESP telemetry JSON into audix_interfaces/msg/EspTelemetry
    exposes the existing /audix/move and /audix/esp/* services
    publishes compatibility /audix/cmd_vel messages for old twist clients

  /audix/gpioHardware
    publishes /audix/ir/state
    exposes buzzer and lift GPIO services

ESP32 micro-ROS firmware
  /audix/esp32_controller
    subscribes /audix/cmd_vel
    subscribes /audix/esp/move_goal
    subscribes /audix/esp/stop_cmd
    publishes /audix/odom
    publishes /audix/esp/telemetry_json
```

The ESP32 PID and mecanum control loop are intentionally preserved. The firmware still runs the original high-priority `controlTask` on core 1 at the same control period. micro-ROS runs separately as `microRosTask` on core 0 and only updates the same command variables that the UART bridge used before.

Firmware

New firmware sketch:

```
ros2_ws/firmware/audix_esp32_microros/audix_esp32_microros.ino
```

The sketch uses `micro_ros_arduino`, `geometry_msgs`, `nav_msgs`, and `std_msgs`. It does not require the custom Audix interfaces on the ESP because the Pi-side `micro_ros_base_node` converts the ESP JSON telemetry into `audix_interfaces`.

Important serial note: once this firmware is flashed, do not run a serial monitor or the old `esp_uart_bridge_node` on the same port. The serial port belongs to `micro_ros_agent`.

Build The ROS Workspace

```
cd ~/pi-testing/ros2_ws
source /opt/ros/humble/setup.bash
rosdep update
rosdep install --from-paths src --ignore-src -y
colcon build --symlink-install
source install/setup.bash
```

Use Jazzy instead of Humble if that is what is installed:

```
source /opt/ros/jazzy/setup.bash
```

Run

Launch the full robot stack with the micro-ROS Agent on the ESP serial port:

```
ros2 launch audix_robot audix_main.launch.py port:=/dev/ttyAMA0 baud:=115200
```

USB serial:

```
ros2 launch audix_robot audix_main.launch.py port:=/dev/ttyUSB0 baud:=115200
```

Base-only launch:

```
ros2 launch audix_robot audix_bridge.launch.py port:=/dev/ttyAMA0 baud:=115200
```

If you want to run the Agent manually:

```
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyAMA0 --baudrate
115200
```

Some installs use this baud syntax instead:

```
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyAMA0 baudrate=115200
```

Standard Velocity Command

The standard ROS drive command is now:

```
/audix/cmd_vel    geometry_msgs/msg/Twist
```

For mecanum drive:

```
linear.x    forward/backward, m/s  
linear.y    left/right strafe, m/s  
angular.z   yaw rate, rad/s
```

Short forward test:

```
ros2 topic pub --rate 10 /audix/cmd_vel geometry_msgs/msg/Twist "{linear: {x:  
0.05, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

Stop:

```
ros2 service call /audix/manager/stop std_srvs/srv/Trigger {}
```

Position Move Service

The manager and dashboard still use the existing Audix position service:

```
/audix/move    audix_interfaces/srv/Move
```

Forward 10 cm:

```
ros2 service call /audix/move audix_interfaces/srv/Move "{angle_deg: 0.0,  
distance_m: 0.10, heading_deg: 0.0, timeout_s: 10.0, wait_for_done: true}"
```

Strafe right 10 cm:

```
ros2 service call /audix/move audix_interfaces/srv/Move "{angle_deg: -90.0,  
distance_m: 0.10, heading_deg: 0.0, timeout_s: 10.0, wait_for_done: true}"
```

The ESP receives this as a ROS topic command on `/audix/esp/move_goal`, not as a UART text command.

Inspect The ROS Graph

List nodes:

```
ros2 node list
```

Expected important nodes:

```
/micro_ros_agent  
/audix/esp32_controller  
/audix/micro_ros_base  
/audix/gpio hardware  
/audix/robot_manager  
/audix/web_dashboard
```

List topics with types:

```
ros2 topic list -t
```

List services with types:

```
ros2 service list -t
```

Show all publishers/subscribers for a node:

```
ros2 node info /audix/esp32_controller  
ros2 node info /audix/micro_ros_base  
ros2 node info /audix/gpio hardware  
ros2 node info /audix/robot_manager
```

Show publishers/subscribers for a topic:

```
ros2 topic info /audix/cmd_vel -v  
ros2 topic info /audix/odom -v  
ros2 topic info /audix/esp/telemetry_json -v  
ros2 topic info /audix/esp/telemetry -v  
ros2 topic info /audix/ir/state -v
```

Echo live telemetry:

```
ros2 topic echo /audix/esp/telemetry  
ros2 topic echo /audix/esp/telemetry_json
```

```
ros2 topic echo /audix/odom
ros2 topic echo /audix/ir/state
```

Show message and service definitions:

```
ros2 interface show geometry_msgs/msg/Twist
ros2 interface show nav_msgs/msg/Odometry
ros2 interface show audix_interfaces/msg/EspTelemetry
ros2 interface show audix_interfaces/srv/Move
```

Safe Test Order

1. Flash `audix_esp32_microros.ino`.
2. Start `audix_bridge.launch.py` with `mock_ir:=true` first if the IR wiring is not ready.
3. Confirm `/audix/esp32_controller` appears in `ros2 node list`.
4. Echo `/audix/esp/telemetry_json`.
5. Echo `/audix/odom`.
6. Call `/audix/esp/reset_odom`.
7. Publish a very small `/audix/cmd_vel` command.
8. Call a 10 cm `/audix/move`.
9. Launch the full `audix_main.launch.py` stack.